

Netty实现心跳与保活特性

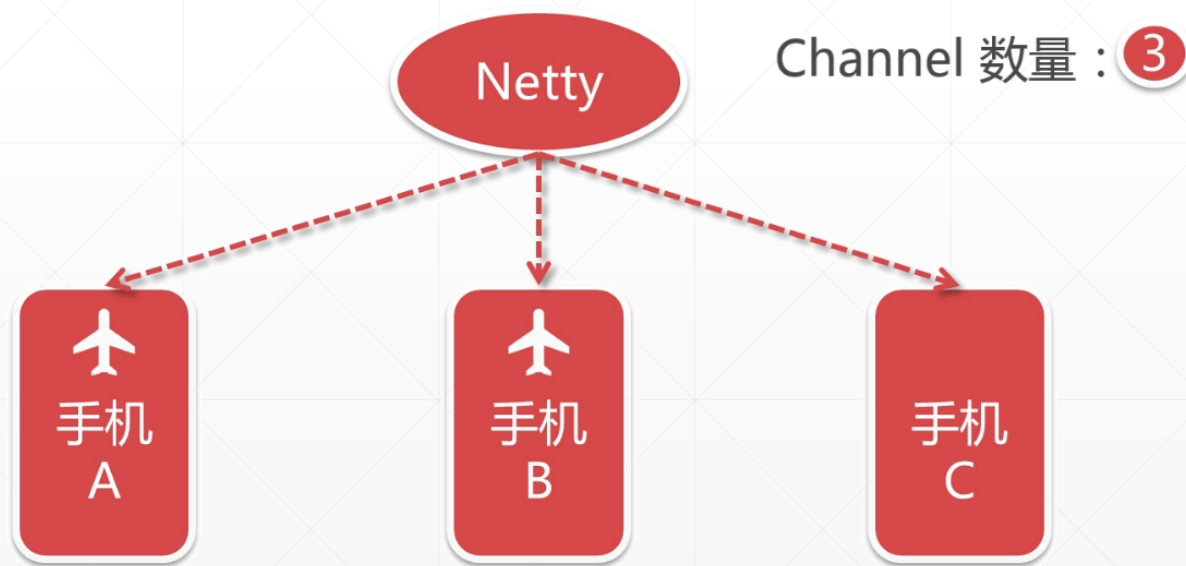
北京理工大学计算机学院
金旭亮



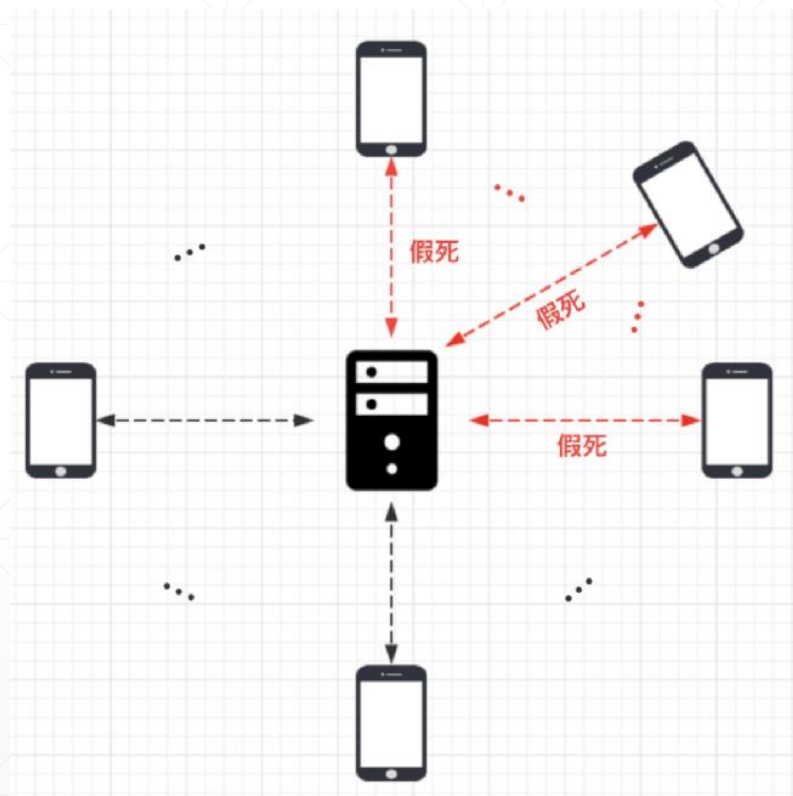
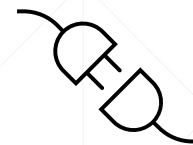
问题描述：手机是经常断开网络连接的.....



手机网络时不时地断开，每次重连，都可能会导致服务端耗费资源以保持一个新的网络连接，而且很可能会出现某个连接客户端其实已经下线，但Server端仍然保持其连接的情况。



“连接假死”在网络应用中很常见



在某一端（服务端或者客户端）看来，底层的TCP连接已经断开，但是应用程序并没有捕获到，因此会认为这条连接仍然是存在的。

通常，连接假死由以下几个原因造成：

1. 应用程序出现线程堵塞，无法进行数据的读写。

2. 客户端或者服务端网络相关的设备出现故障，比如网卡、机房故障。

3. 公网丢包。公网环境相对内网而言，非常容易出现丢包、网络抖动等现象，如果在一段时间内用户接入的网络连续出现丢包现象，那么对客户端来说，数据一直发送不出去；而服务端也一直收不到客户端的数据，连接就一直耗着。

解决方案-心跳机制与空闲检测

1

让客户端定期发送一个心跳包，以便服务端知道特定的客户端还在线。

2

服务端每隔一段时间，检测这段时间内是否有数据读写。



Netty自带的IdleStateHandler就可以实现这个功能。

服务端的空闲检测

第一个Handler是Netty内置的，用于触发“idle（空闲）”事件。第二个Handler则负责响应这些事件，由用户编写。

```
public class MyNettyServer {
    private int port = 9000;
    public void run() throws InterruptedException {
        EventLoopGroup bossGroup = new NioEventLoopGroup();
        EventLoopGroup workerGroup = new NioEventLoopGroup();
        try {
            ServerBootstrap boot = new ServerBootstrap();
            boot.group(bossGroup, workerGroup)
                .channel(NioServerSocketChannel.class)
                .childHandler(new ChannelInitializer<SocketChannel>() {
                    @Override
                    protected void initChannel(SocketChannel socketChannel) throws Exception {
                        //2秒触发一个读空闲事件，3秒触发一个写空闲事件，6秒触发一个读写空闲事件
                        socketChannel.pipeline().addLast(new IdleStateHandler(2, 3, 6));
                        socketChannel.pipeline().addLast(new HeartBeatHandler());
                    }
                });
            ChannelFuture f = boot.bind(port).addListener(future -> {
                if (future.isSuccess()) {
                    System.out.println(port + "端口绑定成功.");
                } else {
                    System.out.println(port + "端口绑定成功失败");
                }
            });
            f.channel().closeFuture().sync();
        } finally {
            bossGroup.shutdownGracefully();
            workerGroup.shutdownGracefully();
        }
    }
}
```


IdleStateHandler及其触发的“通道空闲”事件



IdleStateHandler是Netty内置的一个Handler，它可以指定多长时间内，相关联的Channel没有数据进出时，触发一个IdleStateEvent事件（通过调用fireUserEventTriggered()方法实现的）。



流水线的下一个Handler，可以重写userEventTriggered()方法，在此方法中对IdleStateEvent事件进行捕获，然后写代码处理这个事件。



一种常规的作法，就是等待IdleStateEvent事件触发若干次之后，断开与客户端的连接。



在netty源码中看到fireXXX()，通常就意味着会调用下一个Handler中的XXX()方法。

处理通道空闲事件

```
public class HeartBeatHandler extends ChannelInboundHandlerAdapter {

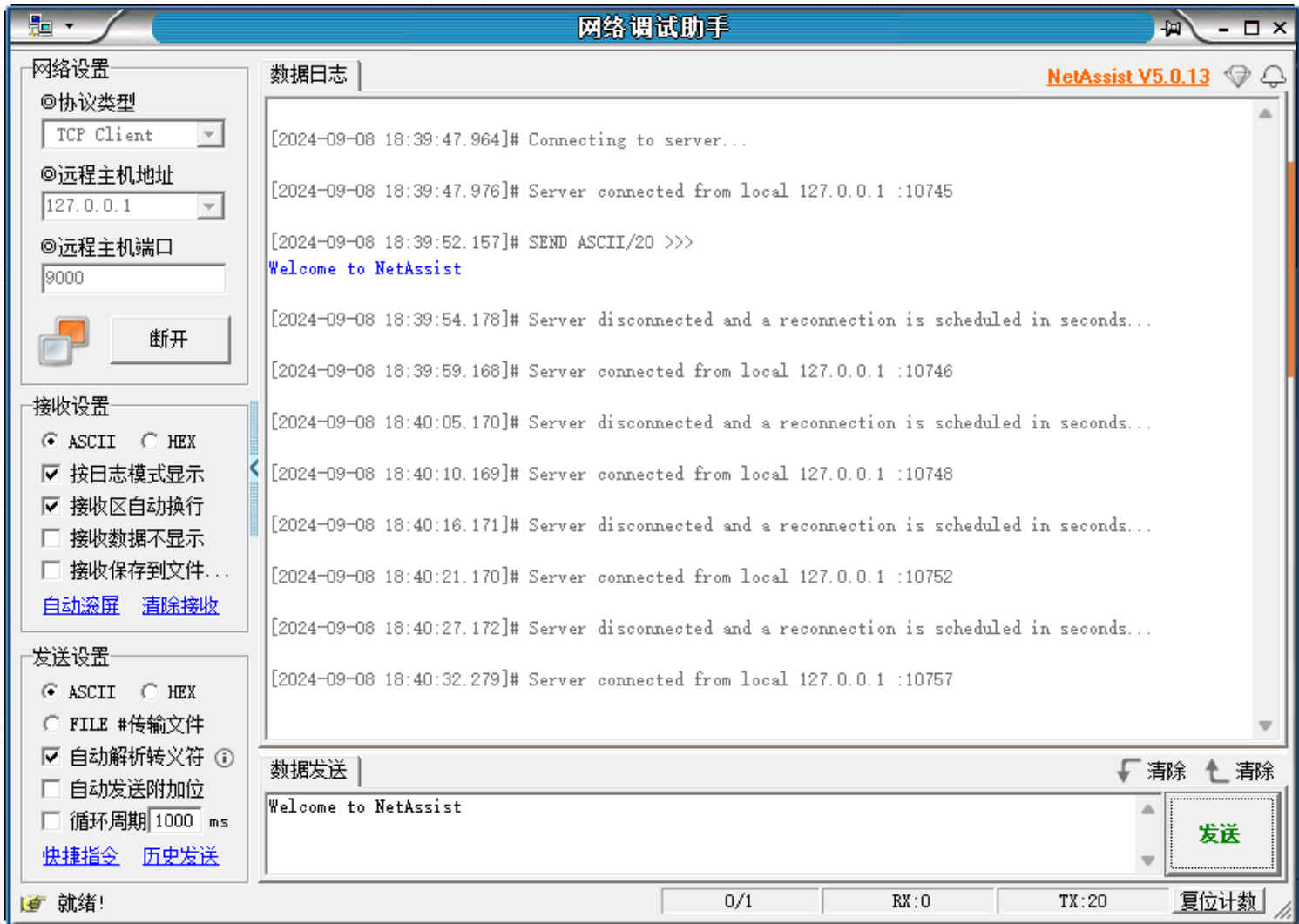
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("有客户端连接: "+ctx.channel().remoteAddress());
    }

    int readTimeout = 0;

    @Override
    public void userEventTriggered(ChannelHandlerContext ctx, Object evt) throws Exception {
        if (evt instanceof IdleStateEvent) {
            IdleStateEvent event = (IdleStateEvent) evt;
            System.out.println("触发事件: " + event.state());
            if (event.state() == IdleState.READER_IDLE) {
                readTimeout++;
            }
            if (readTimeout == 3) {
                System.out.println("超时超过3次, 断开连接");
                ctx.close();
            }
        }
    }
}
```

通常情况下, 可以在Channel中保存上次存取时间, 然后在这里取出来, 检查是否超时, 从而可以进行后继处理。

测试服务端心跳功能



这里使用了一个免费的“网络调试助手”工具，连接上Netty服务器，通过手动控制发送信息的间隔，以便测试服务端的“心跳功能”。

服务端的输出



```
Run MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
有客户端连接: /127.0.0.1:10745
触发事件: READER_IDLE
触发事件: WRITER_IDLE
触发事件: READER_IDLE
触发事件: WRITER_IDLE
触发事件: READER_IDLE
超时超过3次, 断开连接
```

从输出中可以看到，“读/写空闲”事件定时触发，超过3次，程序主动断开连接。

实现心跳技术方案

概述

前面的例子，只是展示了Netty提供的IdleStateHandler的基本用法，现在让我们结合实际，看一下如何在Netty服务端和客户端之间实现“心跳”方案。

1

客户端启动并连接上服务端之后，每隔5秒发送一个内容为“ping”的心跳包给服务端。

2

服务端收到客户端“ping”的心跳包之后，发送一个内容为“pong”的心跳响应包给客户端。

3

客户端连接之后，如果服务端在15秒内没有收到客户端发定的“ping”的心跳包，则会关闭通道。

服务端实现空闲检测



```
public class ServerIdleStateHandler extends IdleStateHandler {
    //15秒必须有心跳数据包发来
    private static final int READER_IDLE_TIME = 15;

    public ServerIdleStateHandler() {
        super(READER_IDLE_TIME, 0, 0, TimeUnit.SECONDS);
    }

    @Override
    protected void channelIdle(ChannelHandlerContext ctx, IdleStateEvent evt) throws Exception {
        System.out.println(READER_IDLE_TIME + "秒内未收到心跳包，关闭连接");
        ctx.channel().close().addListener(future -> {
            if (future.isSuccess() && !ctx.channel().isActive()) {
                System.out.println("客户端通道成功关闭");
            } else {
                System.out.println("客户端通道关闭失败");
            }
        });
    }
}
```

服务端响应pong请求

```
public class ServerPingPongHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("有客户端连接: " + ctx.channel().remoteAddress());
    }

    private static final byte[] pong = "pong".getBytes(StandardCharsets.UTF_8);

    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        ByteBuf bytebuf = (ByteBuf) msg;
        String clientSendMessage = bytebuf.toString(StandardCharsets.UTF_8);
        System.out.println("收到客户端消息: " + clientSendMessage);
        //如果客户端发来的是ping消息
        if (clientSendMessage.equals("ping")) {
            System.out.println(LocalTime.now() + ":服务端发送pong消息。");
            ByteBuf response = Unpooled.wrappedBuffer(pong);
            ctx.writeAndFlush(response);
        }
    }
}
```


服务端核心代码

```
public void run() throws InterruptedException {
    EventLoopGroup bossGroup = new NioEventLoopGroup();
    EventLoopGroup workerGroup = new NioEventLoopGroup();
    try {
        ServerBootstrap boot = new ServerBootstrap();
        boot.group(bossGroup, workerGroup)
            .channel(NioServerSocketChannel.class)
            .childHandler(new ChannelInitializer<SocketChannel>() {
                @Override
                protected void initChannel(SocketChannel socketChannel) throws Exception {
                    socketChannel.pipeline().addLast(new ServerIdleStateHandler());
                    socketChannel.pipeline().addLast(new ServerPingPongHandler());
                }
            });
        ChannelFuture f = boot.bind(port).addListener(future -> {
            if (future.isSuccess()) {
                System.out.println(port + "端口绑定成功.");
            } else {
                System.out.println(port + "端口绑定成功失败");
            }
        });
        f.channel().closeFuture().sync();
    } finally {
        bossGroup.shutdownGracefully();
        workerGroup.shutdownGracefully();
    }
}
```

客户端定期发送心跳包

```
public class ClientHeartBeatHandler extends ChannelInboundHandlerAdapter {
    //客户端每5秒发送一个心跳包
    private static final int HEARTBEAT_INTERVAL = 5;
    private static final byte[] HEARTBEAT_BYTE = "ping".getBytes(StandardCharsets.UTF_8);

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        scheduleSendHeartBeat(ctx, HEARTBEAT_INTERVAL);
        super.channelActive(ctx);
    }

    private void scheduleSendHeartBeat(ChannelHandlerContext ctx, int interval) {
        ctx.executor().schedule(() -> {
            if (ctx.channel().isActive()) {
                System.out.println(LocalTime.now()+":发送心跳包");
                ByteBuf ping = Unpooled.wrappedBuffer(HEARTBEAT_BYTE);
                ctx.writeAndFlush(ping);
                //继续发送心跳包,
                scheduleSendHeartBeat(ctx, interval);
                //如果选择一个较长的时间间隔,比如20秒,服务端会由于在15秒内没收到客户端发来的心跳包
                //而关闭客户端通道
                //scheduleSendHeartBeat(ctx, 20);
            } else {
                System.out.println("通道已关闭");
            }
        }, interval, TimeUnit.SECONDS);
    }
}
```

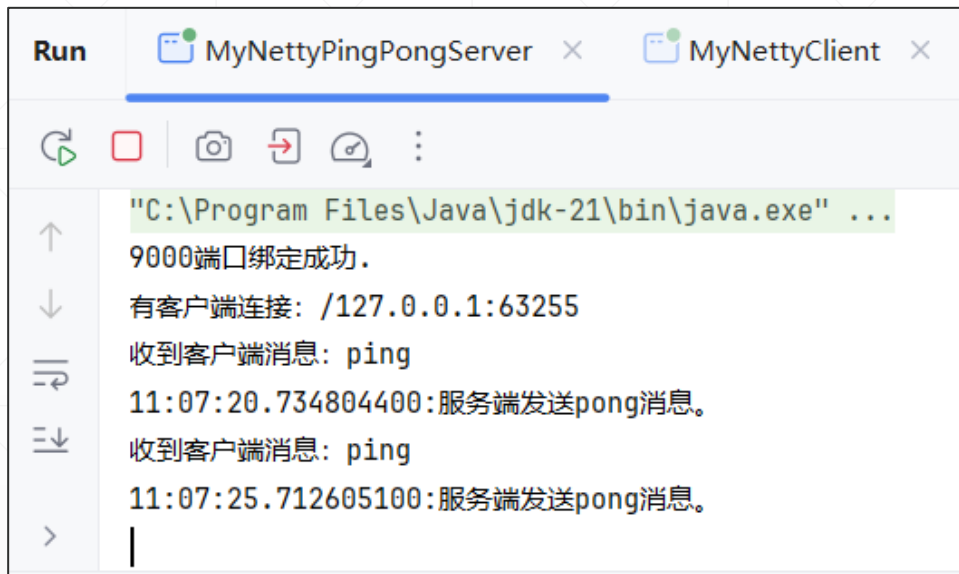
客户端连接服务端并接收响应

```
public class MyClientHandler extends SimpleChannelInboundHandler<Object> {  
    @Override  
    protected void channelRead0(ChannelHandlerContext channelHandlerContext, Object o)  
        throws Exception {  
        ByteBuf buf = (ByteBuf) o;  
        System.out.println("收到服务端响应: "+buf.toString(StandardCharsets.UTF_8));  
    }  
}
```

```
public static void main(String[] args) {  
    NioEventLoopGroup workerGroup = new NioEventLoopGroup();  
    Bootstrap bootstrap = new Bootstrap();  
    bootstrap.group(workerGroup)  
        .channel(NioSocketChannel.class)  
        .handler(new ChannelInitializer<SocketChannel>() {  
            @Override  
            public void initChannel(SocketChannel ch) {  
                ch.pipeline().addLast(new ClientHeartBeatHandler());  
                ch.pipeline().addLast(new MyClientHandler());  
            }  
        });  
    bootstrap.connect("127.0.0.1", 9000).addListener(future -> {  
        if (future.isSuccess()) {  
            System.out.println("连接成功!");  
        } else {  
            System.err.println("连接失败");  
        }  
    });  
}
```

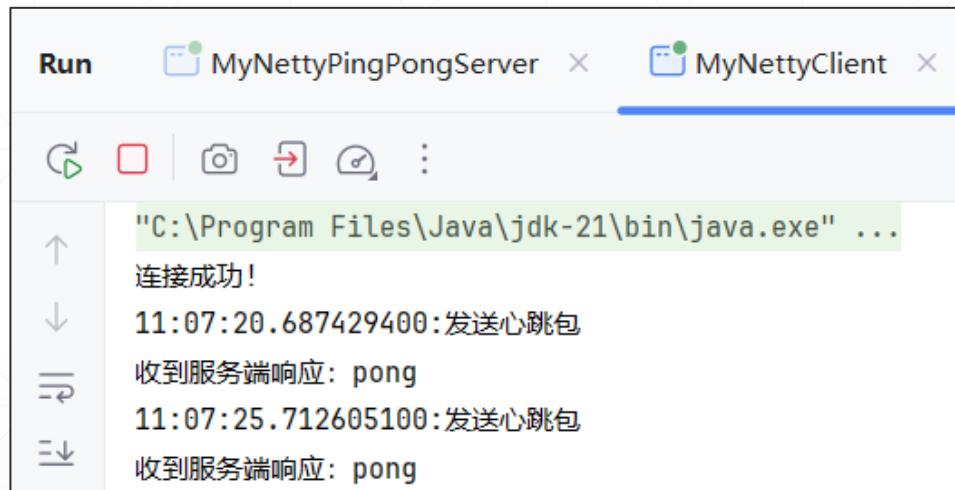
运行截图-正常情况

服务端

A screenshot of a Java IDE's Run console for the 'MyNettyPingPongServer' application. The console shows the following sequence of events: the Java command is executed, port 9000 is successfully bound, a client connects from 127.0.0.1:63255, the server receives a 'ping' message, and then sends a 'pong' message at 11:07:20.734804400. This cycle repeats once more with another 'ping' received and 'pong' sent at 11:07:25.712605100.

```
Run MyNettyPingPongServer x MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
有客户端连接: /127.0.0.1:63255
收到客户端消息: ping
11:07:20.734804400:服务端发送pong消息。
收到客户端消息: ping
11:07:25.712605100:服务端发送pong消息。
|
```

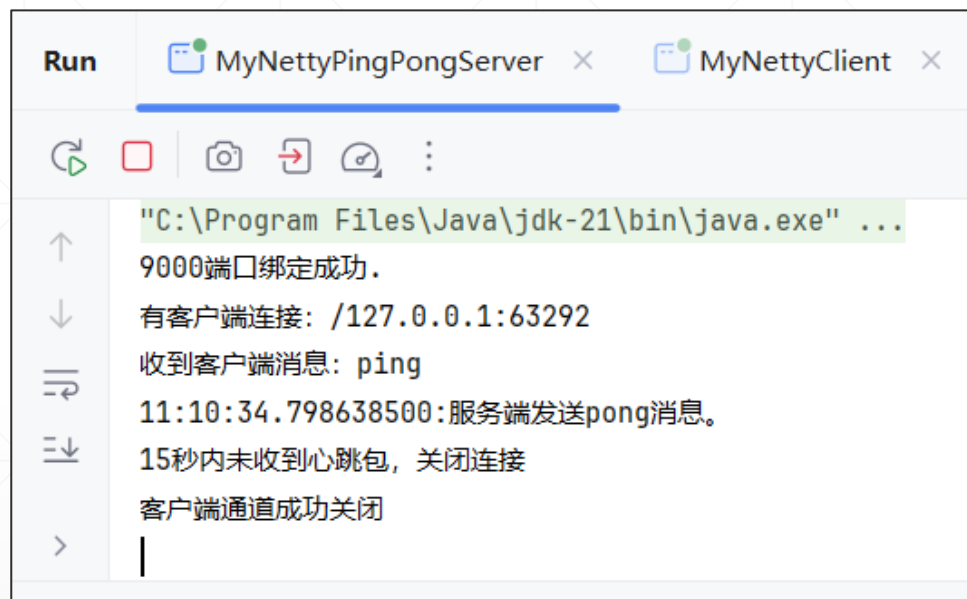
客户端

A screenshot of a Java IDE's Run console for the 'MyNettyClient' application. The console shows the following sequence of events: the Java command is executed, a successful connection is established to the server, a heartbeat packet is sent at 11:07:20.687429400, the server responds with 'pong', another heartbeat packet is sent at 11:07:25.712605100, and the server responds with 'pong' again.

```
Run MyNettyPingPongServer x MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功!
11:07:20.687429400:发送心跳包
收到服务端响应: pong
11:07:25.712605100:发送心跳包
收到服务端响应: pong
```

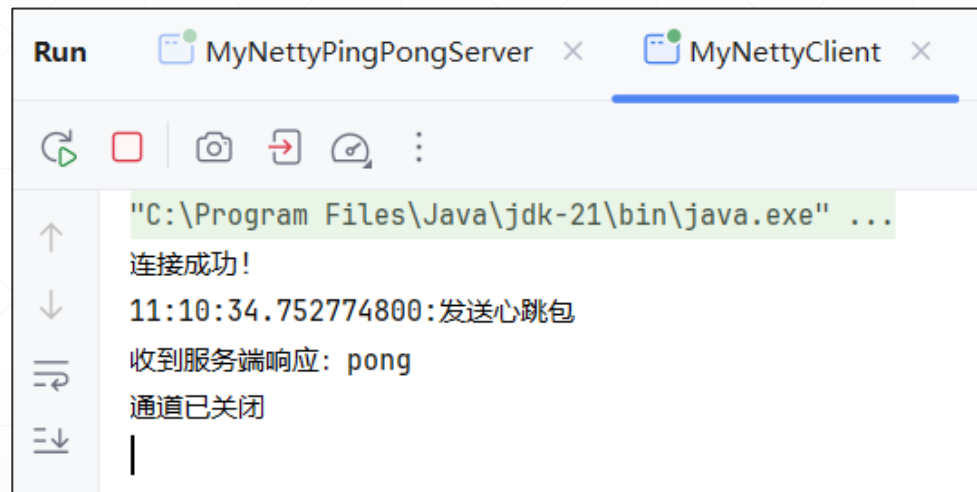
运行截图-假死情况

服务端



```
Run  MyNettyPingPongServer  MyNettyClient
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功.
有客户端连接: /127.0.0.1:63292
收到客户端消息: ping
11:10:34.798638500:服务端发送pong消息.
15秒内未收到心跳包, 关闭连接
客户端通道成功关闭
|
```

客户端



```
Run  MyNettyPingPongServer  MyNettyClient
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功!
11:10:34.752774800:发送心跳包
收到服务端响应: pong
通道已关闭
|
```

由于客户端启用了以下这句, 20秒才发送心跳包:

```
scheduleSendHeartBeat(ctx, 20);
```

导致服务端关闭了通道, 后继请求将无法发送。